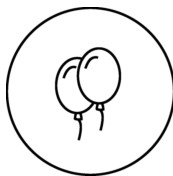


MARATONA DE PROGRAMAÇÃO

CADERNO DE QUESTÕES

Edição 2023



Instruções

LEIA ATENTAMENTE ESTAS INSTRUÇÕES ANTES DE INICIAR A PROVA

Este caderno de tarefas é composto por 10 questões. Verifique se o caderno está completo.

A prova deve ser feita em conjunto apenas pelos membros da equipe.

É proibido consultar a Internet durante a prova.

É permitido consultar qualquer material físico impresso ou manuscrito que a equipe tenha trazido consigo para a prova.

É permitida a consulta ao *help* do ambiente de programação, se este estiver disponível.

Os problemas têm o mesmo valor na correção.

A correção é automatizada, portanto siga atentamente às exigências da tarefa quanto ao formato da entrada e saída de seu programa. Note que até espaços em branco a mais podem resultar em "Presentation error".

Não implemente nenhum recurso gráfico nas suas soluções (janelas, menus, etc.), nem utilize qualquer rotina para limpar a tela ou posicionar o cursor.

Os problemas possuem níveis de dificuldade diversos, de fácil a difícil, ordenadas aleatoriamente; cabe à equipe identificá-los e montar a estratégia mais conveniente.

Preste muita atenção no nome dos arquivos fonte indicados nas tarefas. Soluções na linguagem C devem ser arquivos extensão .c; soluções na linguagem C++ devem ser arquivos com extensão .cpp; soluções na linguagem PHP devem ser arquivos com extensão .php; soluções na linguagem Java devem ser arquivos com extensão .java e a classe principal deve ter o mesmo nome do arquivo fonte; soluções em Python2 ou Python3 devem ter a extensão .py2 ou .py3, respectivamente.

Para problemas diferentes a equipe pode escolher trabalhar com linguagens diferentes; até que a solução para um problema seja aceita, linguagens diferentes podem ser usadas nas submissões de cada problema.

As soluções devem ser enviadas para correção através do formulário disponível na aba **Runs** do ambiente Boca.

Cada submissão errada é penalizada em 20 minutos a mais no tempo total de atividade da equipe; entretanto, o tempo gasto para a solução da questão só é contabilizado uma vez que uma solução é aceita.

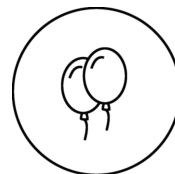
Lembre-se: ganha quem resolver mais questões. O tempo é critério de desempate.

Os dados de entrada devem ser lidos da entrada padrão (teclado) e escritos na saída padrão (tela). Utilize as funções padrão de cada linguagem para entrada e saída de dados.

Observe atentamente as opções de implementação em cada linguagem suportada para os exemplos de aquecimento que seguem na próxima página.

E o mais importante de TUDO: divirtam-se programando!

Boa maratona!



Aposta

Nome do arquivo fonte: aposta.c, aposta.cpp, aposta.java, aposta.php, aposta.js, aposta.py, aposta.py3

Uma turma está empolgada para fazer apostas em uma emocionante partida de futebol. O organizador da aposta será responsável por coletar os palpites para o placar de cada participante, juntamente com o valor da aposta que cada um deseja fazer. Para determinar os vencedores e calcular as premiações, serão aplicados os seguintes critérios de pontuação:

- Pontuação por Acerto do Placar Exato: Os participantes que acertarem o placar exato do jogo receberão 3 pontos.
- Pontuação por Acerto do Vencedor: Além disso, os apostadores que acertarem apenas o vencedor do jogo, mesmo que não acertem o placar exato, receberão 1 ponto.
- Pontuação por Número de gols: Os apostadores que acertarem quantos gols qualquer um dos times fará também receberão 1 ponto, mesmo que errem o vencedor. Esta pontuação poderá se somar à pontuação recebida por acertar o vencedor, mas nunca por acertar o placar exato.

O desafio proposto consiste em desenvolver um programa capaz de calcular a premiação que cada participante irá receber, com base nos critérios mencionados. Para iniciar, o programa solicitará a **quantidade de pessoas** que irão participar da aposta. Em seguida, pedirá o **palpite** de cada apostador, ou seja, a quantidade de gols que cada time fará. Após coletar os palpites, o programa irá requerer o **valor da aposta** que cada participante deseja fazer (com a observação de que todos irão apostar o mesmo valor). Por fim, o programa necessitará da informação sobre o **número de gols** que ambos os times fizeram na partida, a fim de calcular as pontuações e as premiações de acordo com os critérios estabelecidos.

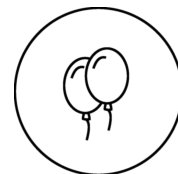
Entrada

A entrada inicia com um número inteiro **N** que indica a quantidade de apostadores. Em seguida, para cada apostador, solicite o número inteiro de gols que o time **A** e **B** farão, seguido de um único valor positivo em ponto flutuante **V** representando o valor da aposta para cada um. Por fim, informe dois números inteiros positivos, **Af** e **Bf**, que representam a quantidade de gols que cada time fez.

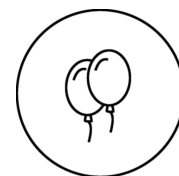
Saída

A saída deve apresentar um número em ponto flutuante, representando o **valor** que cada apostador arrecadou ao final da aposta (incluindo o valor apostado). Os valores de saída devem seguir a mesma ordem dos palpites realizados, devem ser apresentados com duas casas decimais e separados por um espaço.

Entrada	Saída
4 5 3 2 3 1 0 2 8 30 2 3	36.00 48.00 0.00 36.00
5 0 0 3 2 1 3 2 0 2 1 10 3 0	12.50 12.50 0.00 12.50 12.50
6 1 0 2 2 0 3 2 1 1 3 2 0 50 2 2	0.00 140.00 0.00 80.00 0.00 80.00

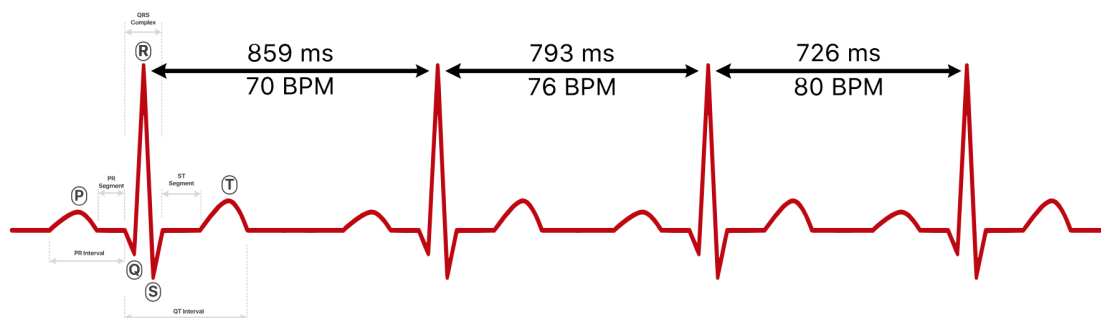


<<PÁGINA INTENCIONALMENTE DEIXADA EM BRANCO>>



BPM - variabilidade da frequência cardíaca

Nome do arquivo fonte: bpm.c, bpm.cpp, bpm.java, bpm.php, bpm.js, bpm.py, bpm.py3



A variabilidade da frequência cardíaca é o fenômeno fisiológico de variação no intervalo de tempo entre os batimentos cardíacos. É medido pela variação no intervalo entre cada batimento do coração. Um método para detectar a variabilidade da frequência cardíaca é utilizando o eletrocardiograma (ECG).

A Figura mostra 2.5 segundos de dados hipotéticos de frequência cardíaca gerados em um ECG, onde são observados 4 batimentos cardíacos completos. Um eletrocardiógrafo digital realiza a aquisição de ECGs a uma frequência fixa (em Hz) e registra os dados em um arquivo, onde cada linha é uma amostra do ECG (linha vermelha da Figura). Por exemplo, em um ECG com frequência de 1000 Hz, cada ponto da curva do ECG representa 1 ms.

A variabilidade da frequência cardíaca é calculada pela distância entre picos R consecutivos. Um pico R é o segmento da onda ECG com maior amplitude. Observando o exemplo da Figura, a distância entre os dois primeiros picos R é 859 ms. Seguindo com a hipótese do registro do ECG a uma frequência de 1000 Hz, é possível afirmar que há 859 pontos entre os dois primeiros picos R.

A unidade de medida utilizada para frequência cardíaca é o BPM -- Batimento por Minuto. Um método simplificado para calcular o BPM é calcular a distância temporal entre dois picos R.

Assumindo que cada ponto do ECG está numa linha de um arquivo, seu objetivo neste problema é calcular todos os BPMs de um ECG. Para realizar esta tarefa são conhecidos: a taxa de amostragem do ECG (frequência em Hz), a quantidade de picos R e a localização de cada pico R no arquivo de ECG.

Entrada

A entrada é dada em três linhas. A primeira linha indica a frequência f (em Hz) do ECG em ponto flutuante. A segunda linha de entrada contém um único número inteiro N ($1 < N < 50$), indicando a quantidade dos picos R. A terceira linha tem N inteiros, em ordem crescente, que correspondem aos números das linhas R ($0 < R_i < 100000$, tal que $1 < i \leq N$) do arquivo de ECG onde cada pico R se encontra.

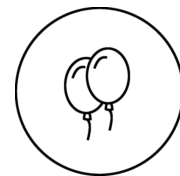
Saída

A saída corresponde aos $N-1$ valores em ponto flutuante de BPM que representam variabilidade da frequência cardíaca.

Entrada	Saída
1000.0 15 283 1204 2159 3188 4211 5188 6200 7232 8200 9157 10156 11198 12159 13139 14162	65.15 62.83 58.31 58.65 61.41 59.29 58.14 61.98 62.70 60.06 57.58 62.43 61.22 58.65
360.0 2	82.13



184 447	
360.0	82.13 80.60 78.26 78.55 75.52 73.22 75.00
13	80.90 80.90 79.12 78.26 76.60
184 447 715 991 1266 1552 1847 2135 2402	
2669 2942 3218 3500	



Ciclismo

Nome do arquivo fonte: ciclismo.c, ciclismo.cpp, ciclismo.java, ciclismo.php, ciclismo.js, ciclismo.py, ciclismo.py3

O JIFS adotará uma nova modalidade do ciclismo para a edição 2024 dos jogos. Trata-se de uma prova de regularidade. Ganha aquele atleta que mais vezes fizer a melhor volta. As baterias têm sempre 4 competidores, e um total de 10 voltas. Um cronômetro geral é disparado no início da bateria e nunca é resetado. Cada bicicleta tem um sensor que dispara na linha de chegada, marcando o tempo do respectivo competidor. Assim, cada competidor terá registrado para si um 10 marcações de tempo. Para saber o tempo de uma volta de um dado competidor, basta subtrair sua medição feita na volta atual pela medição feita na volta anterior.



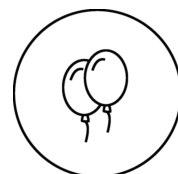
A tarefa é criar um programa que centraliza as medições feitas a cada volta para cada um dos 4 competidores e indicar o vencedor. Ou seja, a cada volta registrada, seu programa deve contabilizar o atleta mais rápido e registrar para ele um ponto. Se dois competidores registram o mesmo melhor tempo na volta, ambos registram ponto. Ao final da corrida, vencerá o competidor que acumulou mais pontos. Contudo, em caso de empate em número de pontos, fica à frente o competidor que terminou a corrida no menor tempo total (que é a marcação da última volta, a volta 10).

Entrada: uma lista de 44 registros de tempo, cada registro composto por 4 números: competidor, minuto [0,59], segundo [0,59] e milésimo de segundo [0,999] registrado a cada volta. Não é necessário validar esses intervalos de tempo. Note que os primeiros 4 registros são sempre com tempo 00:00:000, indicando o início da bateria, e que os demais são sempre incrementais para cada competidor, ainda que possam vir fora de ordem.

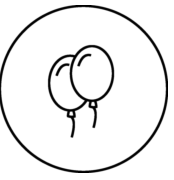
Saída: um único número inteiro entre 1 e 4, indicando o competidor campeão.

Exemplo

Entrada				Saída
1	0	0	0	2
2	0	0	0	
3	0	0	0	
4	0	0	0	
2	1	5	334	
1	1	14	219	
3	1	45	490	
4	1	57	819	



2	2	16	147
1	2	49	580
3	3	15	468
4	3	21	381
2	3	26	763
3	4	19	451
1	4	20	491
2	4	50	600
4	5	8	59
3	5	56	297
2	6	1	624
1	6	8	362
4	6	33	693
1	7	17	276
3	7	21	904
2	7	30	161
4	7	35	476
2	8	51	100
4	9	4	329
1	9	10	1
3	9	11	654
4	10	7	280
2	10	26	234
1	11	0	686
3	11	9	187
4	11	11	773
2	11	36	269
1	12	41	851
2	12	54	312
3	13	0	860
4	13	4	833
1	14	10	111
3	14	44	107
4	14	59	672
1	15	54	610
3	15	58	493



Colisões de Asteroides em Órbita Terrestre

Nome do arquivo fonte: colisoes.c, colisoes.cpp, colisoes.java, colisoes.php, colisoes.js, colisoes.py, colisoes.py3

Você está monitorando um conjunto de cinco asteroides que estão se movendo em órbita ao redor da Terra. Cada asteroide é representado por três números inteiros que representam:

- **tamanho**: número inteiro que indica o tamanho ou massa do asteroide
- **posição**: número inteiro entre 0 e 359 que representa a posição inicial do asteroide na órbita da Terra
- **direção**: número inteiro que indica a direção do movimento do asteroide, onde -1 significa que o asteroide está indo para a esquerda e 1 significa que ele está indo para a direita.

Os asteroides estão se movendo em uma órbita circular de 360 posições, onde cada posição corresponde a um ângulo na órbita da Terra. A cada unidade de tempo, cada asteroide se move em sua direção, avançando ou recuando uma posição na órbita. Este movimento na órbita terrestre faz com que o asteroide se desintegre, ou seja, os asteroides perdem uma unidade de tamanho a cada movimento realizado. Quando dois asteroides ocupam a mesma posição na órbita, ocorre uma colisão. O asteroide que tem o maior tamanho sobrevive e seu tamanho é reduzido subtraindo-se o tamanho do menor asteroide. Se dois asteroides tiverem o mesmo tamanho, ambos são destruídos.

É importante frisar que o movimento do asteroide é independente, deste modo, a cada movimento deve ser testada sua colisão.

Para resolver este problema, você deve escrever um programa que leia a entrada fornecida e determine o estado final dos asteroides após todas as colisões e destruições. O programa deve rodar em um loop até que reste apenas um único asteroide na órbita ou até que todos os asteroides tenham sido destruídos.

- **Entrada**: Uma string representando a entrada no formato descrito, onde cada linha contém três números inteiros separados por espaço, representando o tamanho, a posição e a direção de cada um dos cinco asteroides.

- **Saída**: Uma String que representa o estado final dos asteroides após todas as colisões e destruições no seguinte formato:

Quando houver um asteroide sobrevivente

Sobrevivente

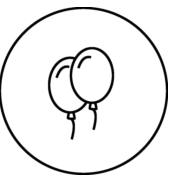
Tamanho: um valor inteiro

Posicao: um valor inteiro

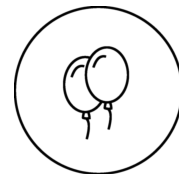
Direcao: um valor inteiro

Quando não houver asteroide sobrevivente

Nenhum asteroide sobrevivente



Entrada	Saída
50 45 1 100 90 -1 30 1 1 80 10 1 90 100 -1	Sobrevivente Tamanho: 328 Posicao: 50 Direcao: -1
500 45 1 100 90 -1 300 135 1 800 49 -1 200 180 1	Sobrevivente Tamanho: 84 Posicao: 272 Direcao: 1
10 321 1 10 83 1 10 135 -1 10 45 -1 10 2 -1	Nenhum asteroide sobrevivente



Figurinhas

Nome do arquivo fonte: figurinhas.c, figurinhas.cpp, figurinhas.java, figurinhas.php, figurinhas.js, figurinhas.py, figurinhas.py3

A cada 4 anos, Ana e João, assim como diversos colegas, iniciam sua coleção de figurinhas para o álbum da copa do mundo. Eles vão adquirindo um conjunto de figurinhas e, com o passar do tempo, diversas figurinhas repetidas começam a surgir, as quais são usadas para realizar trocas entre os amigos. Nesse processo de troca, para evitar conflitos, eles combinaram de trocar sempre a mesma quantidade de figurinhas, e nenhum dos amigos deseja receber figurinhas que não sejam novas.

Para facilitar esse processo, eles gostariam de sua ajuda para criar um aplicativo que facilite o processo de trocas. Para isso você deverá criar um aplicativo que recebe as listas de figurinhas que cada amigo possui, sabendo que cada figurinha possui um número identificador. Ao final, o aplicativo deverá dizer quantas figurinhas podem ser trocadas entre os dois conjuntos repassados, garantindo que ambos recebam a mesma quantidade de figurinhas novas na troca.

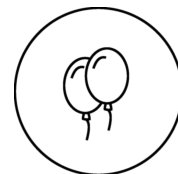
Entrada

A entrada contém, respectivamente, duas sequências numéricas não decrescentes (ou seja, estão ordenadas do menor para o maior) com o número identificador de cada figurinha que Ana e João possuem, onde esses números variam entre 1 e 1000. Assuma que cada amigo pode possuir entre 1 e 10000 figurinhas, incluindo essas quantidades.

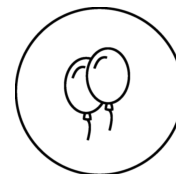
Saída

A saída consiste no quantitativo total de figurinhas a serem trocadas

Entrada	Saída
1 2 3 3 1 2 4	0
1 2 2 2 3 4 4 5 6 7 8 8 8 9 9 10 10	2
1 2 2 2 3 1 3 3 4	0
1 3 3 5 5 5 7 1 3 4 4 4 6 7	1



<<PÁGINA INTENCIONALMENTE DEIXADA EM BRANCO>>



Letroca

Nome do arquivo fonte: letroca.c, letroca.cpp, letroca.java, letroca.php, letroca.js, letroca.py, letroca.py3

Os estudantes de informática do IFSUL corriqueiramente buscam jogos na web para passar o tempo (para os momentos de intervalo das disciplinas). Um desses jogos que foi bastante utilizado pelos estudantes em dado momento foi o Letroca.

O jogo se baseia em entregar uma sequência de letras (desconsideraremos aqui caracteres especiais e acentuação) as quais o usuário deve utilizar (em parte ou em sua totalidade) para tentar formar o maior número de palavras da língua portuguesa que conseguir.

João é muito competitivo e, por isso, deseja contar com sua ajuda para burlar o jogo. Para isso, ele quer que você crie um programa que recebe uma sequência de letras e apresenta na tela em ordem alfabética todas as combinações possíveis para essa sequência de letras repassada, pois João utilizará esse gabarito de combinações para buscar palavras válidas da língua portuguesa que ele conhece dentro do conjunto de saída.

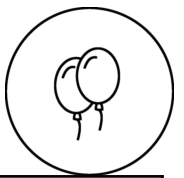
Entrada

A entrada contém uma string contendo a sequência de letras maiúsculas que deverão ser combinadas. Essa sequência de letras varia entre 1 e 5 caracteres. Conforme dito anteriormente, desconsidere caracteres especiais e de acentuação, apenas letras do alfabeto entre A e Z. Não é necessário validar.

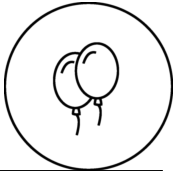
Saída

A saída consiste em várias linhas, contendo todas as sequências de letras que podem ser formadas a partir da sequência de entrada, em ordem alfabética.

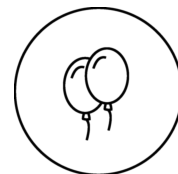
Entrada	Saída
EYES	E EE EES EESY EEY EEYS ES ESE ESEY ESY ESYE EY EYE EYES EYS EYSE S SE SEE SEY SEYE SY SYE SYEE Y YE YEE



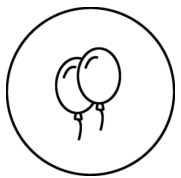
	YEES YES YESE YS YSE YSEE
BLUE	B BE BEL BELU BEU BEUL BL BLE BLEU BLU BLUE BU BUE BUEL BUL BULE E EB EBL EBLU EBU EBUL EL ELB ELBU ELU ELUB EU EUB EUBL EUL EULB L LB LBE LBEU LBU LBUE LE LEB LEBU LEU LEUB LU LUB LUBE LUE LUEB



	U UB UBE UBEL UBL UBLE UE UEB UEBL UEL UELB UL ULB ULBE ULE ULEB
BA	A AB B BA
A	A



<<PÁGINA INTENCIONALMENTE DEIXADA EM BRANCO>>



Índice Pluviométrico

Nome do arquivo fonte: pluviometria.c, pluviometria.cpp, pluviometria.java, pluviometria.php, pluviometria.js, pluviometria.py, pluviometria.py3

Nos últimos meses, o estado do Rio Grande do Sul vem sofrendo com as cheias e inundações por conta do acúmulo de chuvas. Uma das formas de monitorar e prevenir danos materiais e humanos é por meio de medições do índice pluviométrico. Em linhas gerais, o índice pluviométrico refere-se à quantidade de chuva por metro quadrado em determinado local e em determinado período, sendo medido por instrumento chamado de pluviômetro. Uma forma simples, barata e eficiente de se construir um pluviômetro doméstico é utilizando uma garrafa PET.

Nesse problema, vamos considerar que: (i) uma região é monitorada por X pluviômetros domésticos (estação) distribuídos em diferentes locais; (ii) uma região possui um limiar L pré-estabelecido do índice pluviométrico que é utilizado como parâmetro para definir o potencial de risco desta região; e (iii) cada pluviômetro doméstico (estação) desta região possui registros do índice pluviométrico observado nas últimas N horas.

Você deve elaborar um programa que vai ler o número X de pluviômetros domésticos (estação) da região, o limiar L pré-estabelecido do índice pluviométrico da região e o valor N que indica quantas medições (uma por hora) foram realizadas em cada pluviômetro (cada linha na sequência dos dados de entrada representa os índices pluviométricos coletados para cada estação).

O programa deverá informar:

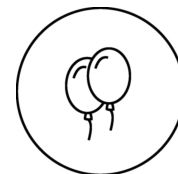
- o acumulado do índice pluviométrico para cada pluviômetro doméstico (estação);
- a média do acumulado dos pluviômetros domésticos que tiveram **entrada válida**;
- o status geral da região, considerando os seguintes parâmetros:
 - [GRANDE PERIGO] se a média do acumulado for maior ou igual a 10% acima do limiar L .
 - [PERIGO] se a média do acumulado for maior ou igual ao limiar L e menor do que 10% acima do limiar L .
 - [PERIGO POTENCIAL] se a média do acumulado for maior ou igual a 80% do limiar L e menor do que o limiar L .
 - [ESTÁVEL] se a média do acumulado estiver abaixo de 80% do limiar L .
- o(s) pluviômetro(s) (estação/ões) com maior(es) acumulado(s), apenas se a média do acumulado seja maior que zero.

Entrada

A primeira linha é um valor inteiro X que representa o número de pluviômetros domésticos (estações) da região. A segunda linha é um valor numérico que representa o limiar L pré-estabelecido do índice pluviométrico da região. A terceira linha é um valor inteiro N que indica quantas medições (uma por hora) foram realizadas em cada pluviômetro doméstico. Na sequência, deverão ter X linhas, contendo N valores numéricos cada uma, separados por espaço em branco.

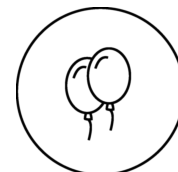
Saída

Para cada pluviômetro doméstico (estação) deverá ser mostrado o acumulado das N medições realizadas. Após, a **média do acumulado** dos pluviômetros caseiros que tiveram **entrada válida**. Caso o número de medições constante para determinado pluviômetro seja diferente de N , o acumulado deve ser desconsiderado da média e deve ser mostrada a mensagem **NÚMERO DE MEDIÇÕES INVÁLIDO**. Na sequência, deve ser mostrado o status da região, conforme a classificação e regras previamente apresentadas. Por último, apresentar o(s) pluviômetro(s)

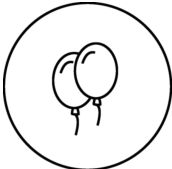


(estação/ões) com maior acumulado, apenas se a média do acumulado seja maior que zero. Caso a entrada tenha um número de linhas de medições diferente de **X**, o programa deverá responder simplesmente **ENTRADA INVÁLIDA**. Caso todas as entradas de cada pluviômetro estejam com **NÚMERO DE MEDIÇÕES INVÁLIDO**, o programa não deverá mostrar a média do acumulado nem o status da região, apresentando simplesmente a mensagem **DADOS INSUFICIENTES!**

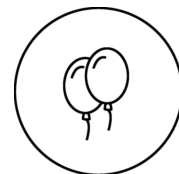
Entrada	Saída
2 40 6 0 0 0 0 0 0 0 0 0 0 0 0	Estacao 1: Acumulado = 0.00mm Estacao 2: Acumulado = 0.00mm ==== MEDIA DO ACUMULADO = 0.00mm STATUS GERAL DA REGIAO = [ESTAVEL]
2 40 6 0 0 3 2 4 1 0 0 2 1 3 0	Estacao 1: Acumulado = 10.00mm Estacao 2: Acumulado = 6.00mm ==== MEDIA DO ACUMULADO = 8.00mm STATUS GERAL DA REGIAO = [ESTAVEL] ==== ESTACOES COM MAIORES ACUMULADOS ESTACAO 1: Acumulado = 10.00mm
2 40 6 0 0 3 2 4 0 0 2 1 3 0	Estacao 1: NÚMERO DE MEDICOES INVALIDO Estacao 2: Acumulado = 6.00mm ==== MEDIA DO ACUMULADO = 6.00mm STATUS GERAL DA REGIAO = [ESTAVEL] ==== ESTACOES COM MAIORES ACUMULADOS ESTACAO 2: Acumulado = 6.00mm
2 40 6 0 0 3 2 4 0 0 2 1 3	Estacao 1: NÚMERO DE MEDICOES INVALIDO Estacao 2: NÚMERO DE MEDICOES INVALIDO DADOS INSUFICIENTES!
2 40 6 0 0 3 2 4 4	ENTRADA INVALIDA!
2 40 6 0 0 10 13.4 35.9 38.7 0 0 12 14.7 32.9 34.7	Estacao 1: Acumulado = 98.00mm Estacao 2: Acumulado = 94.30mm ==== MEDIA DO ACUMULADO = 96.15mm STATUS GERAL DA REGIAO = [GRANDE PERIGO] ==== ESTACOES COM MAIORES ACUMULADOS ESTACAO 1: Acumulado = 98.00mm
2 40 6 10 12 7.8 9 3.3 1.2 9.7 11 7.8 9 3.3 1.2	Estacao 1: Acumulado = 43.30mm Estacao 2: Acumulado = 42.00mm ==== MEDIA DO ACUMULADO = 42.65mm STATUS GERAL DA REGIAO = [PERIGO] ==== ESTACOES COM MAIORES ACUMULADOS ESTACAO 1: Acumulado = 43.30mm
2 40 6 10 12 11 1 3 2 9 10 12 0 2 4	Estacao 1: Acumulado = 39.00mm Estacao 2: Acumulado = 37.00mm ==== MEDIA DO ACUMULADO = 38.00mm STATUS GERAL DA REGIAO = [PERIGO POTENCIAL] ==== ESTACOES COM MAIORES ACUMULADOS ESTACAO 1: Acumulado = 39.00mm



5 80 12 0 0 0 0 0 0 0 0 13 12 46.5 8 0 0 0 0 0 0 0 0 10.9 11.7 43.7 11.8 0 0 0 0 0 0 0 0 11.3 13.2 41.9 12 0 0 0 0 0 0 0 0 12 13 45 9.5 0 0 0 0 0 0 0 0 11 12.3 41 11	Estacao 1: Acumulado = 79.50mm Estacao 2: Acumulado = 78.10mm Estacao 3: Acumulado = 78.40mm Estacao 4: Acumulado = 79.50mm Estacao 5: Acumulado = 75.30mm ==== MEDIA DO ACUMULADO = 78.16mm STATUS GERAL DA REGIAO = [PERIGO POTENCIAL] ==== ESTACOES COM MAIORES ACUMULADOS ESTACAO 1: Acumulado = 79.50mm ESTACAO 4: Acumulado = 79.50mm
5 80 12 0 0 0 0 0 0 0 0 13 12 46.5 8 0 0 0 0 0 0 0 0 10.9 11.7 43.7 11.8 0 0 0 0 0 0 0 0 11.3 13.2 41.9 0 0 0 0 0 0 0 0 12 13 45 9.5 0 0 0 0 0 0 0 0 11 12.3 41 11	Estacao 1: Acumulado = 79.50mm Estacao 2: Acumulado = 78.10mm Estacao 3: NÚMERO DE MEDICOES INVALIDO Estacao 4: Acumulado = 79.50mm Estacao 5: Acumulado = 75.30mm ==== MEDIA DO ACUMULADO = 78.10mm STATUS GERAL DA REGIAO = [PERIGO POTENCIAL] ==== ESTACOES COM MAIORES ACUMULADOS ESTACAO 1: Acumulado = 79.50mm ESTACAO 4: Acumulado = 79.50mm
5 80 12 0 2 0 0 0 3 0 0 13 12 46.5 8 0 0 1 0 0 0 0 0 10.9 11.7 43.7 11.8 0 1 1 1 0 0 0 0 11.3 13.2 41.9 0 0 1 0 0 1 0 0 12 13 45 9.5 0 0 0 2 0 3 0 0 11 12.3 41 11	Estacao 1: Acumulado = 84.50mm Estacao 2: Acumulado = 79.10mm Estacao 3: Acumulado = 69.40mm Estacao 4: Acumulado = 81.50mm Estacao 5: Acumulado = 80.30mm ==== MEDIA DO ACUMULADO = 78.96mm STATUS GERAL DA REGIAO = [PERIGO POTENCIAL] ==== ESTACOES COM MAIORES ACUMULADOS ESTACAO 1: Acumulado = 84.50mm
2 40 6 0 0 0 0 0 0 1 0 0 0 0 0 0	Estacao 1: NÚMERO DE MEDICOES INVALIDO Estacao 2: Acumulado = 0.00mm ==== MEDIA DO ACUMULADO = 0.00mm STATUS GERAL DA REGIAO = [ESTAVEL]
2 40 6 0 0 0 0 0 0 1 0 0 0 0 0 0 2	Estacao 1: NÚMERO DE MEDICOES INVALIDO Estacao 2: NÚMERO DE MEDICOES INVALIDO DADOS INSUFICIENTES!
2 40 6 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1	ENTRADA INVALIDA!
1 40 6 0 0 0 0 1 2	Estacao 1: Acumulado = 3.00mm ==== MEDIA DO ACUMULADO = 3.00mm STATUS GERAL DA REGIAO = [ESTAVEL] ==== ESTACOES COM MAIORES ACUMULADOS ESTACAO 1: Acumulado = 3.00mm
0 40 0	DADOS INSUFICIENTES!
0	ENTRADA INVALIDA!



40	
5	
1 2 3 4 5	



Pódio

Nome do arquivo fonte: podio.c, podio.cpp, podio.java, podio.php, podio.js, podio.py, podio.py3



O pódio é uma plataforma elevada de três níveis diferentes, onde os competidores que melhor se desempenharam são destacados dos demais, para terem sua glória reconhecida por todos. Neste desafio, você deverá implementar um programa que leia o histórico de desempenhos das equipes na modalidade Maratona da Charcode ao longo das últimas 6 edições e indicar quais os 3 melhores desempenhos registrados. Sabe-se que na modalidade Maratona vence quem fizer mais questões. Em caso de empate em número

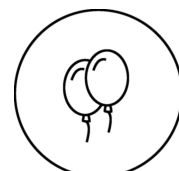
de questões, fica à frente quem levou menos tempo para completar aquele número de questões. Ao final, seu programa deve informar apenas os 3 melhores resultados registrados (número de questões realizadas e tempo gasto).

Entrada: uma lista de registros com 5 inteiros, indicando respectivamente, o número de questões, o número de horas, minutos e segundos registrados para uma dada equipe e o ano em que a equipe competiu. A lista termina quando o último registro contém um número negativo de questões resolvidas.

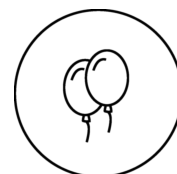
Saída: 3 registros com 5 inteiros cada, indicando o melhor desempenho e o ano, o segundo melhor desempenho e o ano, e o terceiro melhor desempenho e o ano registrados.

Exemplo

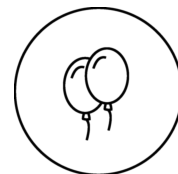
Entrada					Saída				
3	1	52	36	2015	7	2	12	46	2017
2	0	59	2	2015	7	2	32	8	2018
0	0	0	0	2015	7	2	32	22	2018
7	3	40	37	2015					
2	0	37	30	2015					
3	1	2	9	2015					
5	2	39	20	2015					
2	1	25	34	2015					
4	2	18	56	2015					
1	0	22	25	2015					
3	1	32	39	2015					
5	3	30	55	2015					
5	3	34	30	2016					
3	1	22	27	2016					
2	1	12	14	2016					



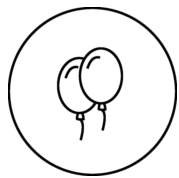
3	1	45	45	2016
3	1	6	54	2016
7	3	36	11	2016
4	1	46	40	2016
4	2	16	36	2016
6	1	55	30	2016
3	2	12	12	2016
3	2	13	36	2016
3	1	47	27	2016
7	3	20	5	2016
6	4	25	42	2016
3	1	23	21	2016
6	3	35	54	2017
6	1	56	0	2017
1	0	15	4	2017
7	2	46	57	2017
1	0	18	16	2017
7	2	12	46	2017
3	1	40	33	2017
4	1	59	4	2017
1	0	18	23	2017
7	5	10	48	2017
2	1	21	36	2017
1	0	21	8	2018
2	1	13	10	2018
7	2	32	8	2018
7	4	36	16	2018
4	1	26	52	2018
0	0	0	0	2018
5	2	41	45	2018
0	0	0	0	2018
2	0	38	36	2018
0	0	0	0	2018
5	1	32	30	2018
2	0	36	18	2018
5	3	27	25	2018
7	3	5	23	2018
7	2	32	22	2018
2	1	14	36	2018
4	1	22	32	2018
4	1	38	36	2019
3	0	57	36	2019
0	0	0	0	2019
0	0	0	0	2019



5	3	2	30	2019
5	2	10	10	2019
2	0	30	2	2019
5	3	1	15	2019
0	0	0	0	2019
1	0	41	9	2019
4	1	24	12	2022
0	0	0	0	2022
6	2	17	54	2022
7	3	10	10	2022
1	0	26	18	2022
0	0	0	0	2022
1	0	29	48	2022
1	0	31	50	2022
5	1	31	10	2022
6	2	20	6	2022
0	0	0	0	2022
6	3	15	30	2022
4	2	40	16	2022
3	1	59	45	2022



<<PÁGINA INTENCIONALMENTE DEIXADA EM BRANCO>>



Resta 1

Nome do arquivo fonte: resta1.c, resta1.cpp, resta1.java, resta1.php, resta1.js, resta1.py, resta1.py3

RESTA 1 é um tradicional jogo de tabuleiro que contém inicialmente 32 peças, tendo como objetivo fazer com que apenas uma peça, dentre as trinta e duas, permaneça no tabuleiro. Nesse jogo, são permitidos apenas movimentos de saltar, horizontal ou verticalmente, sobre uma peça em posição adjacente (imediatamente vizinha), desde que a posição destino esteja vazia (peças não podem ser sobrepostas). Considere a matriz abaixo como uma representação inicial do tabuleiro do RESTA 1. Observe que as casas com X são posições proibidas no jogo. As casas com o número 1 são posições que estão preenchidas e a casa com número 0 é uma posição vazia.

		colunas						
		0	1	2	3	4	5	6
linhas	0	X	X	1	1	1	X	X
	1	X	X	1	1	1	X	X
	2	1	1	1	1	1	1	1
	3	1	1	1	0	1	1	1
	4	1	1	1	1	1	1	1
	5	X	X	1	1	1	X	X
	6	X	X	1	1	1	X	X

Agora você lembrou do funcionamento do jogo RESTA 1, elabore um programa que recebe como entrada uma determinada configuração do tabuleiro e responde quais são as jogadas possíveis de serem feitas.

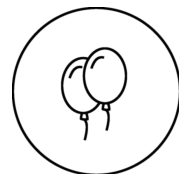
Entrada

A entrada será constituída por 7 linhas e, em cada linha, terão 7 valores separados por 1 espaço em branco. Considere que todas as entradas estarão preenchidas com X nas casas proibidas do tabuleiro (conforme exemplo apresentado acima) e as demais casas estarão obrigatoriamente preenchidas com 1 (um) ou 0 (zero), sendo que o 1 (um) representa uma casa preenchida e o 0 (zero) representa uma casa vazia. O tabuleiro terá no máximo 32 casas preenchidas com peças.

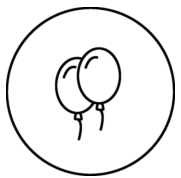
Saída

Caso tabuleiro não tenha peças, o programa deverá escrever a mensagem “TABULEIRO VAZIO!”. Caso o tabuleiro tenha apenas uma peça, o programa deverá escrever a mensagem “RESTA UM!”. Caso o tabuleiro tenha mais de duas peças, o programa deverá mostrar o número de jogadas possíveis e quais são elas.

Entrada	Saída
X X 1 1 1 X X X X 1 1 1 X X 1 1 1 1 1 1 1 1 1 1 0 1 1 1 1 1 1 1 1 1 1 X X 1 1 1 X X X X 1 1 1 X X	JOGADAS POSSIVEIS: 4 MOVA [1][3] PARA [3][3] MOVA [3][1] PARA [3][3] MOVA [3][5] PARA [3][3] MOVA [5][3] PARA [3][3]



X X 0 0 0 X X X X 0 0 0 X X 0 X X 0 0 0 X X X X 0 0 0 X X	TABULEIRO VAZIO!
X X 0 0 0 X X X X 0 0 0 X X 0 X X 0 0 0 X X X X 0 0 1 X X	RESTA UM!
X X 1 0 1 X X X X 0 0 0 X X 1 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 0 1 X X 0 0 0 X X X X 1 0 1 X X	JOGADAS POSSIVEIS: 0
X X 1 0 1 X X X X 0 0 0 X X 1 0 0 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 0 0 1 X X 0 0 0 X X X X 1 0 1 X X	JOGADAS POSSIVEIS: 0
X X 1 0 1 X X X X 0 0 0 X X 1 0 0 0 0 0 1 0 0 1 1 0 0 0 1 0 0 0 0 0 1 X X 0 0 0 X X X X 1 0 1 X X	JOGADAS POSSIVEIS: 2 MOVA [3][2] PARA [3][4] MOVA [3][3] PARA [3][1]
X X 1 0 1 X X X X 0 1 0 X X 1 1 0 1 0 0 1 1 0 1 1 0 0 0 0 0 0 1 0 0 1 X X 0 0 0 X X X X 1 0 1 X X	JOGADAS POSSIVEIS: 6 MOVA [2][0] PARA [4][0] MOVA [2][0] PARA [2][2] MOVA [2][3] PARA [0][3] MOVA [3][2] PARA [3][4] MOVA [3][3] PARA [5][3] MOVA [3][3] PARA [3][1]
X X 1 0 1 X X X X 0 1 0 X X 1 1 0 1 0 0 1 1 0 1 1 0 1 1 0 0 0 1 0 0 1 X X 0 0 0 X X X X 1 0 1 X X	JOGADAS POSSIVEIS: 7 MOVA [2][0] PARA [4][0] MOVA [2][0] PARA [2][2] MOVA [2][3] PARA [0][3] MOVA [3][2] PARA [3][4] MOVA [3][3] PARA [5][3] MOVA [3][3] PARA [3][1] MOVA [3][6] PARA [3][4]
X X 1 1 1 X X X X 1 1 1 X X 1 1 1 1 1 1 1 1 0 0 1 1 1 1 1 1 1 1 1 1 1 X X 1 1 1 X X X X 1 1 1 X X	JOGADAS POSSIVEIS: 3 MOVA [1][2] PARA [3][2] MOVA [3][4] PARA [3][2] MOVA [5][2] PARA [3][2]



Pedra, Papel e Tesoura

Nome do arquivo fonte: rps.c, rps.cpp, rps.java, rps.php, rps.js, rps.py, rps.py3

Certamente você já jogou Pedra, Papel e Tesoura! Originalmente conhecido como *Jankenpon*, esse é um jogo de origem japonesa, muito simples e popular, normalmente utilizado com método de seleção (da mesma forma que o par ou ímpar, jogo de dados e moeda). No *Jankenpon*, cada jogador estica simultaneamente uma das mãos, apresentando um símbolo, conforme explicado abaixo:

- A pedra é simbolizada pelo punho fechado.
- O papel pelos cinco dedos da mão esticados.
- A tesoura pelo dedo indicador e pelo dedo do meio esticados.

Para saber quem ganhou, são utilizadas as regras abaixo:

- **Pedra** ganha da **tesoura** (amassando-a ou quebrando-a).
- **Tesoura** ganha do **papel** (cortando-o).
- **Papel** ganha da **pedra** (embrulhando-a).

Se ambos os jogadores utilizam o mesmo símbolo, é considerado empate. Sabendo dessas regras, elabore um programa que solicite o símbolo de dois jogadores e responda quem ganhou a partida de *Jankenpon*. Para representar os três símbolos, você deve utilizar a seguinte convenção numérica abaixo. Caso, algum dos jogadores utilize uma numeração diferente das listadas abaixo, o programa deverá dizer que o jogo foi inválido.

- 0 (zero) significa **Pedra**.
- 2 (dois) significa **Tesoura**.
- 5 (cinco) significa **Papel**.

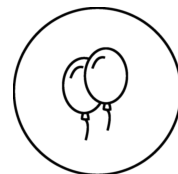
Entrada

A primeira terá um número inteiro que representa o símbolo do JOGADOR 1 e a segunda linha terá um número inteiro que representa o símbolo do JOGADOR 2.

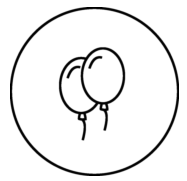
Saída

Deverá ser mostrado: JOGADOR 1 se o ganhador for o número representado na primeira linha; JOGADOR 2 se o ganhador for o número representado na segunda linha; EMPATE caso os dois jogadores utilizem o mesmo símbolo; ou JOGO INVALIDO caso algum dos jogadores utilize um número diferente da convenção definida previamente.

Entrada	Saída
0 2	JOGADOR 1
2 5	JOGADOR 1
0 5	JOGADOR 2
2 2	EMPATE
2 1	JOGO INVALIDO
7	JOGO INVALIDO



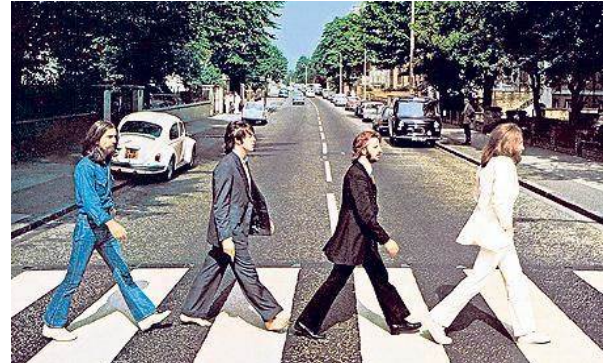
5	
2	JOGADOR 2
0	
9	JOGO INVALIDO
9	



Semáforo

Nome do arquivo fonte: semaforo.c, semaforo.cpp, semaforo.java, semaforo.php, semaforo.js, semaforo.py, semaforo.py3

Em uma cidade movimentada, onde o trânsito de pedestres é intenso, agentes de trânsito estão empenhados em otimizar o funcionamento de um semáforo específico em uma rua crucial. O objetivo é minimizar o tempo durante o qual o semáforo permanece fechado para os carros, garantindo, ao mesmo tempo, que a maioria dos pedestres tenha a oportunidade de atravessar a rua com segurança.



Para alcançar esse objetivo, o agente de trânsito está conduzindo um estudo minucioso do fluxo de pedestres nessa rua. A cada ciclo em que o semáforo está fechado, ele registra a quantidade de pedestres que atravessam a rua e o tempo que cada um leva para fazê-lo. O objetivo principal é determinar o menor número de segundos que o semáforo deve permanecer aberto para que pelo menos um percentual específico, representado por X%, dos pedestres tenha a oportunidade de atravessar a rua.

A tarefa que se propõe é a seguinte: desenvolver um programa que receba como entrada o número de observações feitas pelo agente de trânsito. Cada observação corresponde a um ciclo de semáforo aberto para pedestres que foi analisado. Para cada ciclo, o programa deve fornecer o número de pessoas que atravessaram a rua durante aquele período e o tempo que cada uma delas levou, expresso em números inteiros medidos em segundos. Ao final do programa, o usuário deve indicar o percentual alvo mínimo de pedestres que se deseja que consiga atravessar a rua, representado também como um número inteiro.

Exemplo:

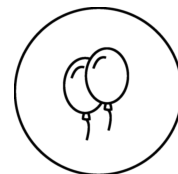
- 3 ciclos estudados.
- Ciclo 1: 3 pedestres.
 - Tempos do ciclo 1: 12, 10, 12 segundos.
- Ciclo 2: 5 pedestres.
 - Tempos do ciclo 2: 11, 13, 10, 14, 11 segundos.
- Ciclo 3: 4 pedestres.
 - Tempos do ciclo 3: 16, 11, 13, 8 segundos.
- Percentual alvo: 90%.

Os tempos totais dos pedestres foram: 12 10 12 11 13 10 14 11 16 11 13 8

Caso o semáforo seja definido em 14 segundos, 11 dos 12 pedestres conseguirão atravessar (91,66%). Porém, caso seja definido em 13 segundos, somente 10 dos 12 pedestres (83,33%) conseguirão atravessar. Portanto, o menor valor para assegurar que **90%** dos pedestres consigam atravessar é **14** segundos.

Entrada

A entrada inicia com um número inteiro de ciclos **C** analisados. Em seguida, para cada ciclo **C**, um número inteiro **N** representando o número de pessoas que atravessaram a rua naquele ciclo e para cada **N**, o número de segundos **T** que cada pessoa demorou para atravessar a rua, representado



como um número inteiro. Por fim, um número inteiro **P**, que representa o percentual alvo mínimo de pessoas que irão conseguir atravessar a rua para o tempo calculado.

Saída

A saída deve apresentar um número inteiro que representa o tempo que deve ser configurado o semáforo.

Entrada	Saída
3 3 12 10 12 5 11 13 10 14 11 4 16 11 13 8 90	14
4 2 13 15 4 11 10 9 8 6 14 12 13 9 10 8 1 17 70	13
2 7 11 10 13 12 9 8 14 6 12 13 10 9 11 8 50	11